

Pai Theory Assignment#2

Arshia Azeem 24k-0012 BSAI-3A

Task1:

```
import requests
```

with open("api.txt",'r') as file:

```
apiUrl=file.read()
```

try:

```
response=requests.get(apiUrl)
```

```
if response.status_code==200:
```

```
data=response.json()
```

```
if data.get("status")=="success" and "data" in data and data["data"]:
```

```
matchlist=data['data']
```

```
print("API Status:")
```

```
print(data['info'])
```

```
for match in matchlist:
```

```
print("-----")
```

```
print(f'Match Name: {match['name']}')
```

```
print(f'Match Status: {match['status']}')
```

```
print(f'venue: {match['venue']}')
```

```
print(f'Date: {match['date']}')
```

```
if 'score' in match:
```

```
for score in match['score']:
```

```
print()
```

```
print(f'Inning: {score["inning"]}')
```

```
print(f'Runs: {score['r']}')
```

```
        print(f'Wickets: {score['w']}')
        print(f'Overs: {score['o']}')
    else:
        errorMessage=data.get("reason","No live matches found or an unknown error
occurred.")
        print(f"Could not retrieve matches:{errorMessage}")

    else:
        print(f"Error: Failed to fetch data. Status Code:{response.status_code}")
        print("Response:",response.text)
except requests.exceptions.RequestException as e:
    print(f"A network error occurred:{e}")
```

```
> python -u "c:\Users\DELL\Desktop\pyth\A2 task1.py"
```

API Status:

```
{'hitsToday': 9, 'hitsUsed': 1, 'hitsLimit': 100, 'credits': 0, 'server': 1, 'offsetRows': 0, 'totalRows': 6, 'queryTime': 24.731, 's': 0, 'cache': 1}
```

```
-----  
Match Name: Bangladesh vs Ireland, 1st Test  
Match Status: Day 1: Stumps - Ireland opt to bat  
venue: Sylhet International Cricket Stadium, Sylhet  
Date: 2025-11-11
```

```
Inning: ireland Inning 1  
Runs: 270  
Wickets: 8  
Overs: 90  
-----
```

```
Match Name: Pakistan vs Sri Lanka, 1st ODI  
Match Status: Pakistan won by 6 runs  
venue: Rawalpindi Cricket Stadium, Rawalpindi  
Date: 2025-11-11
```

```
Inning: Pakistan Inning 1  
Runs: 299  
Wickets: 5  
Overs: 50
```

```
Inning: Sri Lanka Inning 1  
Runs: 293  
Wickets: 9  
Overs: 50  
-----
```

```
Match Name: Western Australia vs Queensland, 12th Match  
Match Status: Day 1: Stumps - Western Australia opt to bowl  
venue: W.A.C.A. Ground, Perth  
Date: 2025-11-11
```

```
Inning: Queensland Inning 1  
Runs: 323  
Wickets: 6  
Overs: 91  
-----
```

```
Match Name: Tasmania vs South Australia, 11th Match  
Match Status: Day 2: Stumps - Tasmania lead by 209 runs  
venue: Bellerive Oval, Hobart  
Date: 2025-11-09
```

```
Inning: Tasmania Inning 1  
Runs: 209  
Wickets: 10  
Overs: 64.2
```

```
Inning: South Australia Inning 1  
Runs: 177  
Wickets: 10  
Overs: 48
```

```

Inning: Tasmania Inning 2
Runs: 177
Wickets: 9
Overs: 50
-----
Match Name: New South Wales vs Victoria, 10th Match
Match Status: Day 2: Stumps - Victoria lead by 310 runs
venue: Sydney Cricket Ground, Sydney
Date: 2025-11-09

Inning: victoria Inning 1
Runs: 382
Wickets: 10
Overs: 103.1

Inning: New South Wales,Victoria Inning 1
Runs: 128
Wickets: 10
Overs: 49.5

Inning: victoria Inning 2
Runs: 56
Wickets: 2
Overs: 24
-----
Match Name: Melbourne Renegades Women vs Sydney Thunder Women, 5th Match
Match Status: Melbourne Renegades Women won by 4 wkts
venue: Junction Oval, Melbourne
Date: 2025-11-11

Inning: Sydney Thunder Women Inning 1
Runs: 148
Wickets: 7
Overs: 20

Inning: Melbourne Renegades Women Inning 1
Runs: 151
Wickets: 6
Overs: 18.1
PS C:\Users\DELL\Desktop\pyth>

```

Task2:

```

import numpy as np

import csv

data=[]

with open("sensor_data.csv","r") as file:

    for line in file:

        row=line.strip().split(",")

        data.append(row)

arr=np.array(data,dtype=float)

arr[arr==-999]=np.nan

arr[(arr<0)|(arr>100)]=np.nan

```

```

sensor_mean=np.nanmean(arr,axis=0)

hour_median=np.nanmedian(arr,axis=1)

inv_count=np.isnan(arr).sum(axis=0)

worst_sensor=np.argmax(inv_count)

min_val=np.nanmin(arr)

max_val=np.nanmax(arr)

arr_norm=(arr-min_val)/(max_val-min_val)

np.savetxt("sensor_data_normalized.csv",arr_norm,delimiter=",")

print("Worst sensor index: ",worst_sensor)

```

```

> python -u "c:\Users\DELL\
Desktop\pyth\A2 task2.py"
Worst sensor index: 83
PS C:\Users\DELL\Desktop\pyth>

```

```

1 1.40124999999999996e-01,7.10749999999999929e-01,4.716249999999999609e-01,9.0700000000000000e-01,
2 4.6550000000000000249e-01,7.71249999999999911e-01,7.304999999999999272e-01,1.7749999999999999
3 3.633750000000000036e-01,1.481250000000000067e-01,2.73249999999999929e-01,5.3600000000000000e-01,
4 5.7487500000000000249e-01,8.400000000000000799e-01,4.53625000000000004e-01,4.0874999999999999
5 8.342499999999999361e-01,7.763750000000000373e-01,nan,2.14499999999999962e-01,5.6862500000000000e-01,
6 9.859999999999999876e-01,2.62624999999999973e-01,nan,8.058750000000000080e-01,5.7000000000000000e-01,
7 2.798749999999999849e-01,4.425000000000000044e-01,9.7500000000000000333e-02,1.7299999999999999
8 8.4637499999999998774e-01,2.808749999999999858e-01,nan,8.8487500000000000782e-01,9.8475000000000000e-01,
9 6.3475000000000000364e-01,2.610000000000000098e-01,4.2575000000000000173e-01,1.7037499999999999
10 7.079999999999999627e-01,5.3462500000000000169e-01,6.8887500000000000151e-01,3.3287500000000000e-01,
11 8.962499999999999911e-01,2.28374999999999947e-01,5.110000000000000098e-01,5.3849999999999999
12 9.32500000000000001343e-02,3.130000000000000004e-01,nan,7.033749999999999725e-01,8.7100000000000000e-01,
13 nan,4.8500000000000000838e-02,3.964999999999999636e-01,3.50000000000000001027e-02,6.5287499999999999
14 8.9300000000000000160e-01,2.5237500000000000160e-01,nan,3.5387500000000000506e-01,8.5799999999999999
15 2.8150000000000000280e-01,4.137500000000000067e-01,1.486250000000000071e-01,4.8912500000000000e-01,
16 1.696249999999999980e-01,5.3400000000000000302e-01,8.5474999999999998987e-01,6.3625000000000000e-01,
17 6.2837500000000000169e-01,8.5362500000000000782e-01,3.5975000000000000142e-01,5.4249999999999999
18 nan,3.1737500000000000182e-01,4.2300000000000000426e-01,8.883749999999999147e-01,7.9149999999999999
19 2.281249999999999944e-01,4.2175000000000000138e-01,1.8475000000000000253e-01,8.3474999999999999
20 6.923749999999999627e-01,4.8625000000000000849e-02,9.149999999999999800e-02,6.9100000000000000e-01,
21 9.976249999999999840e-01,9.4249999999999998657e-02,9.199999999999999289e-01,nan,4.2350000000000000e-01,
22 5.9625000000000000577e-01,7.112499999999999378e-02,nan,5.224999999999999784e-02,8.4824999999999999
23 nan,7.321249999999999147e-01,9.6249999999999994671e-03,1.1175000000000000160e-01,1.6875000000000000e-01,

```

Task3:

```
import pandas as pd

df=pd.read_csv("Titanic-Dataset.csv")

report=[]

for col in df.columns:

    dtype=df[col].dtype

    missing=df[col].isnull().sum()

    percentage=(missing/len(df))*100

    report.append(f"{col}: dtype={dtype}, missing={missing}({percentage:.2f}%)")

with open("inspect_report.txt","w") as f:

    f.write("\n".join(report))


df['Age'] = df.groupby(['Pclass','Sex'])['Age'].transform(lambda x: x.fillna(x.median()))

embarked_mode=df['Embarked'].mode()[0]

df['Embarked']=df['Embarked'].fillna(embarked_mode)

df=df.drop(columns=['Cabin'])

df['FamilySize']=df['SibSp']+df['Parch']

df['IsAlone']=(df['FamilySize']==0).astype(int)

df['Age']=df['Age'].astype('int64')

df.to_csv("titanic_cleaned.csv",index=False)
```

```
≡ inspect_report.txt
1  PassengerId: dtype=int64, missing=0(0.00%)
2  Survived: dtype=int64, missing=0(0.00%)
3  Pclass: dtype=int64, missing=0(0.00%)
4  Name: dtype=object, missing=0(0.00%)
5  Sex: dtype=object, missing=0(0.00%)
6  Age: dtype=float64, missing=177(19.87%)
7  SibSp: dtype=int64, missing=0(0.00%)
8  Parch: dtype=int64, missing=0(0.00%)
9  Ticket: dtype=object, missing=0(0.00%)
10 Fare: dtype=float64, missing=0(0.00%)
11 Cabin: dtype=object, missing=687(77.10%)
12 Embarked: dtype=object, missing=2(0.22%)
```

Task4:

```
import pandas as pd
```

```
df=pd.read_csv("titanic_cleaned.csv")
```

```
fares=pd.read_csv("ticket_fares.csv")
```

```
df = df.merge(fares,on="Ticket",how="left")
```

```
df.columns = df.columns.str.strip()
```

```
if "Fare_y" in df.columns:
```

```
    df["Fare"] = df["Fare_y"]
```

```
bins=[0,12,19,59,120]; labels=['Child','Teen','Adult','Senior']
```

```
df['AgeGroup']=pd.cut(df['Age'],bins=bins,labels=labels, right=True)
```

```
survival_by_sex_age=df.pivot_table(values="Survived",index="Sex",
```

```
columns="AgeGroup",aggfunc="mean")
```

```
print(survival_by_sex_age)
```

```
survival_by_sex_age.to_csv("survival_by_sex_age.csv")
```

```
class_survival=df.groupby("Pclass")["Survived"].mean()
```

```
print(class_survival)
```

```
class_survival.to_csv("class_survival.csv")
```

```
df["FareBin"]=pd.qcut(df["Fare"],q=4, labels=["Low","Medium","High","VeryHigh"])
```

```
fare_survival=df.groupby("FareBin")["Survived"].mean()
```

```
print(fare_survival)
```

```
fare_survival.to_csv("fare_survival.csv")
```

```
with open("report.txt","w") as f:
```

```
    f.write("Hypothesis 1: Women and children first.\n")
```

```
    f.write("Based on the survival rates grouped by sex and age group, women show a  
substantially higher survival rate than men across all age groups. Children, particularly  
female children, also show higher survival rates compared to adult males. These results  
support the hypothesis that 'women and children first' influenced survival outcomes.\n\n")
```

```
    f.write("Hypothesis 2: The wealthy had a higher survival rate.\n")
```

```
    f.write("Both methods used to approximate wealth—Passenger Class and Fare quantile  
bins—show that passengers in higher classes and those who paid higher fares had  
significantly higher survival rates. Therefore, the data supports the hypothesis that  
wealthier passengers were more likely to survive.\n")
```



```

blems (Ctrl+Shift+M) LL\Desktop\pyth> python -u "c:\Users\DELL\Desktop\pyth\A2 task4.py"
c:\Users\DELL\Desktop\pyth\A2 task4.py:14: FutureWarning: The default value of observed=False is deprecated and will change to observed=True in a future version of pandas. Specify observed=False to silence this warning and retain the current behavior
  survival_by_sex_age=df.pivot_table(values="Survived",index="Sex",
AgeGroup  Child    Teen    Adult    Senior
Sex
female    0.38835  0.753247  0.699561  1.000000
male      0.32000  0.086957  0.202624  0.111111
Pclass
1         0.689055
2         0.480000
3         0.239057
Name: Survived, dtype: float64
c:\Users\DELL\Desktop\pyth\A2 task4.py:24: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
  fare_survival=df.groupby("FareBin")["Survived"].mean()
FareBin
Low        0.229426
Medium     0.441687
High       0.336449
VeryHigh   0.609418
Name: Survived, dtype: float64
PS C:\Users\DELL\Desktop\pyth> 

```

≡ report.txt

```

1 Hypothesis 1: Women and children first.
2 Based on the survival rates grouped by sex and age group, women show a
3 substantially higher survival rate than men across all age groups. Children,
4 particularly female children, also show higher survival rates compared to adult males.
5 These results support the hypothesis that 'women and children first' influenced survival
6 outcomes.
7
8 Hypothesis 2: The wealthy had a higher survival rate.
9 Both methods used to approximate wealth Passenger Class and Fare quantile
10 bins show that passengers in higher classes and those who paid higher fares
11 had significantly higher survival rates. Therefore, the data supports the hypothesis
12 that wealthier passengers were more likely to survive.

```